



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería en Software

EXPOSICIÓN - DEMOSTRACIÓN

DESARROLLO DIRIGIDO POR MODELOS (MDD): GENERACIÓN DE CÓDIGO FUENTE A PARTIR DE MODELOS UML

Proyecto Fin de Curso:

MundiPets

Asignatura:

Ingeniería de Requisitos

Curso y paralelo:

4to Software "B"

Docente:

Ing. GUERRERO ULLOA GLEISTON CICERON

Integrantes:

MORALES SÁNCHEZ GARY ALEJANDRO CI: 1251154173 gmoraless2@uteq.edu.ec Rol:
responsable del GitHub, modelado UML, verificación de código, roundtrip

NIEVES SÁNCHEZ JIMMY SAMUEL CI: 1250907878 jnievess@uteq.edu.ec Rol:
documentador, configuración de Java Designer, marco teórico, generación, compilación

Repositorio GitHub:

<https://github.com/MoralesGaryUteq/PFC-MundiPets-MDD-EquipoE>

Periodo académico:

2026-2027 PPA

Índice general

RESUMEN	2
1. MARCO TEÓRICO	3
2. SELECCIÓN DE LA HERRAMIENTA MDD	5
3. DESCRIPCIÓN DEL SISTEMA MUNDIPETS	7
4. MODELO UML DE ORIGEN	9
5. CONFIGURACIÓN Y GENERACIÓN DEL CÓDIGO FUENTE	11
6. VERIFICACIÓN DEL SOFTWARE GENERADO	13
7. ROUNDTRIP ENGINEERING	15
8. REPOSITORIO GITHUB Y TRABAJO COLABORATIVO	17
9. CONCLUSIONES	18
10.REFERENCIAS	19
11.ANEXOS	20

RESUMEN

El presente proyecto demuestra la aplicación del enfoque *Model Driven Development* (MDD) en el subsistema de gestión de información de mascotas y control veterinario de MundiPets. Para ello, se construyó un modelo UML de clases en Modelio 5.4.01 y se utilizó el módulo Java Designer 5.4.00 para generar automáticamente estructuras de código fuente Java.

El proceso comprendió la definición y validación del modelo UML, la aplicación del estereotipo «**JavaClass**», la generación de archivos Java, la compilación del código obtenido y la verificación de la correspondencia entre los elementos del modelo y los artefactos generados.

Los resultados evidenciaron que la precisión de los atributos, operaciones, parámetros, asociaciones y multiplicidades del modelo influye directamente en la calidad del código producido. Después de corregir las inconsistencias identificadas y regenerar las clases, el código compiló correctamente, demostrando la trazabilidad estructural entre el modelo UML y la implementación Java.

1 MARCO TEÓRICO

1.1 Ingeniería y Desarrollo Dirigidos por Modelos

La Ingeniería Dirigida por Modelos (*Model Driven Engineering*, MDE) es un enfoque de desarrollo de software en el cual los modelos constituyen artefactos principales del proceso. A diferencia del modelado utilizado únicamente como documentación, en MDE los modelos contienen información estructurada que puede ser procesada mediante mecanismos automáticos de transformación [1].

El Desarrollo Dirigido por Modelos (*Model Driven Development*, MDD) representa la aplicación práctica de estos principios. En este enfoque, el sistema se describe mediante modelos formales que posteriormente se transforman en código fuente u otros artefactos de implementación. Su finalidad es aumentar el nivel de abstracción, reducir tareas repetitivas y mantener la trazabilidad entre el diseño y el software generado [2].

Aunque MDD facilita la automatización del desarrollo, todavía presenta desafíos relacionados con la interoperabilidad de herramientas, la evolución de los modelos, la mantenibilidad y su adopción en proyectos reales [3].

1.2 Arquitectura Dirigida por Modelos

La Arquitectura Dirigida por Modelos (*Model Driven Architecture*, MDA) es una propuesta del Object Management Group que organiza el desarrollo mediante diferentes niveles de abstracción [4].

Los principales niveles son:

- **Modelo Independiente de Computación (CIM):** representa los procesos del negocio y las necesidades de los usuarios sin considerar aspectos tecnológicos.
- **Modelo Independiente de Plataforma (PIM):** describe la estructura y el comportamiento del sistema sin depender de una plataforma específica.
- **Modelo Específico de Plataforma (PSM):** incorpora las características tecnológicas necesarias para implementar el sistema en una plataforma determinada.

En el proyecto MundiPets, el diagrama de clases UML representa la estructura del subsistema y se utiliza como modelo de origen para generar código Java mediante Java Designer.

1.3 UML, perfiles y estereotipos

El Lenguaje Unificado de Modelado (*Unified Modeling Language*, UML) es un estándar utilizado para especificar, visualizar y documentar sistemas de software. Sus diagramas permiten representar clases, atributos, operaciones, relaciones y multiplicidades dentro de una solución orientada a objetos [5].

Los perfiles UML permiten adaptar el lenguaje a necesidades tecnológicas específicas mediante estereotipos y valores etiquetados. En MundiPets se empleó el estereotipo:

«JavaClass»

Este estereotipo, proporcionado por Java Designer 5.4.00, permite identificar las clases que deben transformarse en archivos Java. No se utilizaron estereotipos de persistencia, servicios o controladores, porque el generador empleado produce principalmente estructuras de clases Java.

Los modelos y perfiles se sustentan en metamodelos que definen las reglas de construcción de los elementos utilizados. El estándar Meta Object Facility proporciona una infraestructura para definir metamodelos y facilitar la interoperabilidad entre herramientas dirigidas por modelos [6].

1.4 Transformaciones y generación de código

Las transformaciones de modelos permiten convertir una representación del sistema en otra. Dentro de MDD se distinguen principalmente dos tipos:

- **Modelo a Modelo (M2M):** transforma un modelo de origen en otro modelo con diferente nivel de abstracción. Un ejemplo es la transformación de un PIM hacia un PSM. El estándar QVT proporciona mecanismos para definir este tipo de transformaciones [7].
- **Modelo a Texto (M2T):** transforma elementos de un modelo en archivos textuales, como código fuente. El estándar MOFM2T establece mecanismos para generar texto a partir de modelos basados en MOF [8].

La generación realizada en MundiPets corresponde a una transformación M2T, debido a que Java Designer interpreta las clases, atributos, operaciones, asociaciones y multiplicidades del modelo UML para producir archivos con extensión `.java`.

La ingeniería directa describe el proceso de generar código desde un modelo. Por otra parte, el roundtrip controlado permite modificar el modelo y regenerar el código para comprobar que el cambio se refleja en el artefacto correspondiente. En este proyecto se añadió el atributo `microchip` a la clase `Mascota` y posteriormente se regeneró `Mascota.java`.

La generación automática disminuye parte de la codificación repetitiva, pero no garantiza por sí sola una aplicación completa. La lógica de negocio, la persistencia, la interfaz gráfica y otros componentes deben ser implementados manualmente. Por ello, el código generado debe complementarse con procesos de validación, compilación y prueba.

2 SELECCIÓN DE LA HERRAMIENTA MDD

2.1 Herramienta seleccionada

Para el desarrollo del proyecto MundiPets se seleccionó **Modelio 5.4.01** como entorno de modelado UML y **Java Designer 5.4.00** como módulo de generación de código Java.

De acuerdo con la documentación oficial, Modelio es un entorno de modelado orientado al desarrollo de software que proporciona soporte para UML, asistencia durante el modelado, comprobación de consistencia y una arquitectura extensible mediante módulos [9].

Dentro de esta arquitectura, Java Designer permite generar código Java desde un modelo UML, recuperar información desde código existente, generar documentación Javadoc y automatizar determinadas actividades relacionadas con el desarrollo Java [10].

La versión utilizada corresponde a la distribución de código abierto de Modelio, publicada bajo la licencia **GNU General Public License versión 3.0 (GPL-3.0)** [11]. Por esta razón, su utilización académica no requirió adquirir una licencia comercial, lo cual favoreció la reproducibilidad de la demostración por parte de los integrantes y del docente.

2.2 Características principales

Las principales características de Modelio relacionadas con el desarrollo dirigido por modelos son:

- **Soporte para UML:** permite elaborar diagramas estructurales y de comportamiento utilizando el estándar UML definido por el Object Management Group (OMG), facilitando la representación formal del sistema mediante clases, atributos, operaciones y relaciones [5].
- **Generación automática de código:** mediante módulos especializados permite transformar elementos del modelo UML en estructuras iniciales de código fuente, aplicando transformaciones modelo-a-texto dentro del proceso MDD [8].
- **Uso de estereotipos UML:** permite extender los elementos del modelo mediante información adicional asociada a perfiles UML, facilitando la adaptación del modelo hacia plataformas específicas [5].
- **Ingeniería directa e inversa:** permite establecer mecanismos para generar código desde modelos y recuperar modelos desde código existente, favoreciendo la evolución y mantenimiento del software [2].
- **Arquitectura modular:** incorpora módulos adicionales que amplían las capacidades de la herramienta, permitiendo incorporar funcionalidades de generación, validación y personalización del proceso de desarrollo.

2.3 Justificación de la herramienta

La selección de Modelio 5.4.01 con Java Designer 5.4.00 se realizó considerando cinco criterios: soporte UML, generación de código Java, validación del modelo, licencia y facilidad de reproducción del proceso.

Modelio permite construir diagramas UML y ejecutar comprobaciones de consistencia sobre los elementos del modelo. Estas funciones fueron necesarias para representar las clases, atributos, operaciones, asociaciones y multiplicidades del subsistema MundiPets antes de la generación del código [9].

Java Designer fue seleccionado porque constituye el módulo específico de Modelio para trabajar con Java. La documentación oficial señala que este módulo soporta generación de código, ingeniería inversa, documentación Javadoc y automatización Java [10]. En la demostración se utilizó su configuración estándar para transformar las clases UML marcadas con el estereotipo «JavaClass» en archivos con extensión `.java`.

Otro criterio determinante fue el licenciamiento. Modelio y sus extensiones de código abierto se distribuyen bajo la licencia **GPL-3.0** [11]. En consecuencia, el equipo pudo instalar, utilizar y documentar la herramienta sin adquirir una licencia comercial, condición apropiada para un proyecto académico y para la reproducción posterior de la demostración.

Frente a las alternativas analizadas, Modelio ofreció una combinación adecuada de modelado UML, auditoría, generación Java y disponibilidad sin costo de licencia. Enterprise Architect y determinadas funciones de Visual Paradigm dependen de ediciones comerciales, mientras que Papyrus con Acceleo permite mayor personalización, pero requiere una configuración más compleja. Por estas razones, Modelio fue la alternativa más adecuada para demostrar el flujo UML → Java Designer → código Java dentro del alcance de MundiPets.

2.4 Comparación con herramientas alternativas

La selección de una herramienta MDD requiere analizar diferentes alternativas considerando aspectos como soporte UML, capacidad de transformación, generación de código, licencia y facilidad de uso. La siguiente comparación permite justificar la elección realizada.

Cuadro 2.1: Comparación de herramientas MDD alternativas

Herramienta	Ventajas	Limitaciones
Modelio 5.4.01	Soporte UML, auditoría del modelo, arquitectura modular, Java Designer para generación de código Java y distribución de código abierto bajo licencia GPL-3.0 [9-11].	El generador estándar produce principalmente estructuras de clases; la lógica de negocio y otras funcionalidades deben completarse manualmente.
Enterprise Architect	Amplio soporte MDA, múltiples lenguajes de programación y generación de código mediante plantillas.	Requiere licencia comercial para varias funcionalidades avanzadas.
Visual Paradigm	Amplias capacidades de modelado empresarial e integración tecnológica, con soporte para diferentes diagramas UML.	Varias funciones relacionadas con generación y transformación dependen de versiones comerciales.
Papyrus + Acceleo	Alta personalización mediante estándares OMG y soporte para transformaciones modelo a texto (M2T).	Mayor complejidad de configuración y aprendizaje inicial.

Después del análisis comparativo, Modelio fue seleccionado debido a que proporciona las funcionalidades necesarias para cumplir los objetivos del proyecto MundiPets, permitiendo desarrollar el flujo completo de Ingeniería Dirigida por Modelos desde la construcción del modelo UML hasta la generación del código fuente.

3 DESCRIPCIÓN DEL SISTEMA MUNDIPETS

El proyecto MundiPets responde a una necesidad frecuente en el desarrollo tradicional de software: la separación entre las etapas de diseño e implementación, que suele producir inconsistencias entre los modelos documentados y el código construido manualmente. Para reducir esta brecha, el proyecto aplica el enfoque *Model Driven Development* (MDD), utilizando un modelo UML como fuente autoritativa a partir de la cual se genera y regenera el código fuente de forma trazable.

3.1 Descripción general del sistema

MundiPets es una plataforma digital orientada a la gestión responsable de mascotas, cuyo propósito es facilitar la administración de información relacionada con animales de compañía, procesos de adopción, seguimiento sanitario y comunicación entre los diferentes actores involucrados.

El sistema permite centralizar información relevante de las mascotas mediante el registro de datos generales, historial médico, vacunas, documentos veterinarios y antecedentes asociados. De esta manera, proporciona una estructura organizada para la consulta y gestión de información necesaria durante los procesos de cuidado, adopción y seguimiento veterinario.

Desde la perspectiva del Desarrollo Dirigido por Modelos (*Model Driven Development, MDD*), MundiPets utiliza modelos UML como elementos principales para representar la estructura del sistema antes de la generación del código fuente. El modelo de clases desarrollado permite definir las entidades principales del dominio, sus atributos, operaciones y relaciones, sirviendo como base para el proceso de transformación automática mediante la herramienta seleccionada.

3.2 Alcance del módulo desarrollado

Para la demostración del proceso MDD se desarrolló el módulo de gestión de información de mascotas y control veterinario, debido a que permite representar adecuadamente los elementos principales del dominio y evidenciar la transformación desde un modelo UML hacia código fuente.

El módulo contempla las siguientes funcionalidades:

- Registro de información básica de mascotas, incluyendo nombre, raza, sexo, fecha de nacimiento y estado de salud.
- Administración del historial médico, permitiendo almacenar información relacionada con enfermedades, tratamientos y antecedentes clínicos.
- Gestión de vacunas mediante el registro de nombre de vacuna, fecha de aplicación y fecha de vencimiento.
- Gestión de documentos veterinarios asociados a la mascota, permitiendo almacenar y validar información médica.
- Asociación entre mascotas y profesionales veterinarios para la validación de información sanitaria.

El alcance del desarrollo se enfoca en la construcción del modelo estructural del sistema mediante UML y su posterior transformación hacia código fuente utilizando la herramienta Modelio 5.4 con el módulo Java Designer.

3.3 Actores del subsistema

Los actores directamente relacionados con el subsistema de gestión de información de mascotas y control veterinario son:

- **Propietario:** usuario responsable de registrar, actualizar y consultar la información general y sanitaria de las mascotas asociadas a su perfil.
- **Veterinario:** profesional encargado de consultar información médica, revisar documentos veterinarios y registrar validaciones relacionadas con la salud de las mascotas.

La clase **Mascota** constituye la entidad principal del dominio, pero no representa un actor externo del sistema.

Estos requisitos funcionales representan la base para la construcción del modelo UML del sistema MundiPets, donde cada elemento del dominio es representado mediante clases, atributos, métodos y relaciones. Posteriormente, el modelo será utilizado como entrada para el proceso de generación automática de código dentro del enfoque MDD.

3.4 Requisitos funcionales relevantes

El subsistema modelado implementa los requisitos funcionales relacionados con la gestión y validación médica de la mascota, definidos originalmente en la Especificación de Requisitos de Software (ERS) del proyecto MundiPets. La Tabla 3.1 resume dichos requisitos y los vincula con la clase del modelo UML que los implementa.

Cuadro 3.1: Requisitos funcionales cubiertos por el subsistema modelado

ID	Nombre	Descripción	Clase(s) relacionada(s)
RF-02	Gestionar historial médico de la mascota	Registrar y actualizar el historial médico de la mascota (enfermedades, tratamientos, controles y observaciones).	HistorialMedico
RF-03	Consultar perfil completo de una mascota	Visualizar la información general y el historial médico autorizado de una mascota.	Mascota
RF-09	Validar la información médica de una mascota	Permitir que un veterinario autorizado valide la información médica registrada, marcándola como verificada.	Validacion-Medica, Veterinario
RF-10	Gestionar recordatorios de controles preventivos	Generar recordatorios de próximas vacunas, desparasitaciones y controles a partir de las fechas registradas.	Vacuna, HistorialMedico
RF-14	Gestionar carnet de vacunación	Registrar, actualizar y consultar el carnet de vacunación, incluyendo fechas de aplicación y próximas dosis.	Vacuna
RF-16	Gestionar antecedentes genéticos y parentesco	Registrar y consultar antecedentes genéticos, enfermedades hereditarias y grado de parentesco de la mascota.	Antecedente-Genetico

Estos requisitos funcionales representan la base para la construcción del modelo UML del sistema MundiPets, donde cada elemento del dominio es representado mediante clases, atributos, métodos y relaciones. Posteriormente, el modelo será utilizado como entrada para el proceso de generación automática de código dentro del enfoque MDD.

4 MODELO UML DE ORIGEN

4.1 Diagrama y clases del subsistema

El modelo UML del subsistema de gestión de información de mascotas y control veterinario fue desarrollado en **Modelio 5.4.01**. El diagrama de clases constituye el modelo de origen utilizado por **Java Designer 5.4.00** para generar la estructura del código Java.

El modelo contiene ocho clases: **Propietario**, **Mascota**, **HistorialMedico**, **Vacuna**, **DocumentoVeterinario**, **Veterinario**, **ValidacionMedica** y **AntecedenteGenetico**. Todas poseen atributos tipados, operaciones, visibilidad y relaciones con multiplicidades definidas.

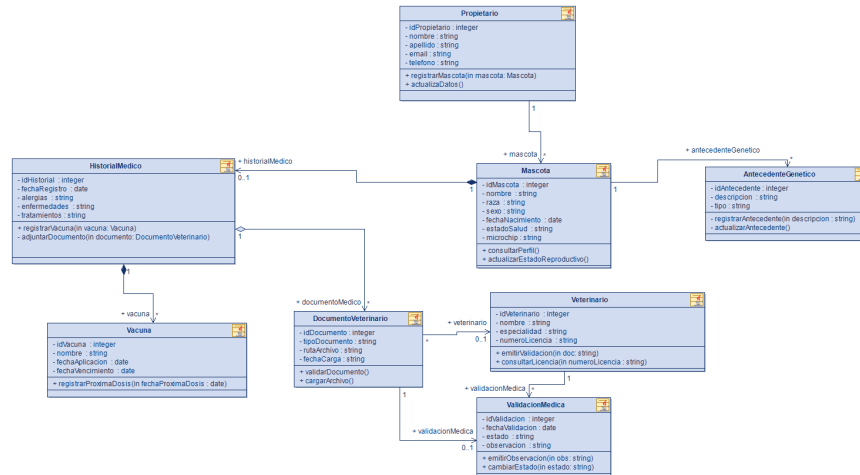


Figura 4.1: Diagrama de clases UML del subsistema MundiPets.

Las responsabilidades principales se resumen en la Tabla 4.1.

Cuadro 4.1: Responsabilidades de las clases

Clase	Responsabilidad
Propietario	Registrar y administrar las mascotas de su perfil.
Mascota	Centralizar los datos generales y sanitarios del animal.
HistorialMedico	Organizar enfermedades, tratamientos y registros médicos.
Vacuna	Registrar vacunas y sus fechas de aplicación y vencimiento.
DocumentoVeterinario	Representar certificados, informes y archivos médicos.
Veterinario	Consultar información y emitir validaciones profesionales.
ValidacionMedica	Registrar el estado y las observaciones de una validación.
AntecedenteGenetico	Registrar antecedentes y riesgos hereditarios.

4.2 Relaciones del modelo

El diagrama incorpora asociaciones, agregaciones y composiciones. Las multiplicidades indican la cantidad de objetos que participan en cada relación y permiten que Java Designer genere referencias simples o colecciones tipadas.

- **Propietario–Mascota:** asociación uno a muchos. Cada mascota pertenece a un propietario (1) y un propietario puede administrar cero o varias mascotas (0..*).
- **Mascota–HistorialMedico:** composición. Una mascota puede tener un historial (0..1) y cada historial pertenece a una mascota (1).
- **Mascota–AntecedenteGenetico:** asociación. Una mascota puede registrar varios antecedentes (0..*) y cada antecedente pertenece a una mascota (1).
- **HistorialMedico–Vacuna:** composición. Un historial puede contener varias vacunas (0..*) y cada vacuna pertenece a un historial (1).
- **HistorialMedico–DocumentoVeterinario:** agregación. Un historial puede reunir varios documentos (0..*) y cada documento puede asociarse con un historial (0..1).

- **Veterinario–DocumentoVeterinario:** asociación. Un veterinario puede revisar varios documentos (0..*) y cada documento puede relacionarse con un veterinario (0..1).
- **Veterinario–ValidacionMedica:** asociación. Un veterinario puede emitir varias validaciones (0..*) y cada validación corresponde a un veterinario (1).
- **DocumentoVeterinario–ValidacionMedica:** asociación. Un documento puede tener una validación (0..1) y cada validación se relaciona con un documento (1).

Las relaciones múltiples pueden transformarse en colecciones como `List<Mascota>`, `List<Vacuna>` o `List<ValidacionMedica>`. El modelo no incorpora generalización, debido a que `Propietario` y `Veterinario` se representan como clases independientes.

4.3 Estereotipo y validación

Para habilitar la generación se utilizó el perfil de **Java Designer 5.4.00** y se aplicó a las ocho clases el estereotipo:

«JavaClass»

No se emplearon los estereotipos «Entity», «Service» ni «Controller», porque el generador produce estructuras Java y no implementa automáticamente persistencia, servicios o una arquitectura por capas.

La validación se realizó seleccionando el paquete principal y ejecutando:

Audit → Check Model

El resultado registró **cero errores, cero advertencias y cero recomendaciones**.

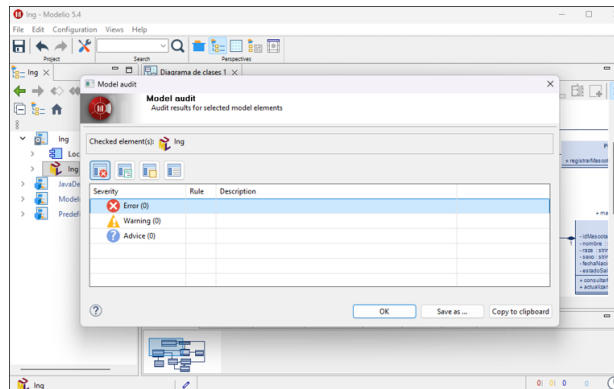


Figura 4.2: Resultado de la auditoría automática del modelo UML.

La auditoría se complementó con una revisión manual de tipos, firmas de operaciones, multiplicidades, nombres de rol y estereotipos, debido a que algunos problemas solo se manifiestan durante la generación o compilación del código.

4.4 Archivo nativo del modelo

El proyecto nativo de Modelio, el diagrama UML y las evidencias se encuentran almacenados en el repositorio público:

Repositorio del modelo MundiPets en GitHub

5 CONFIGURACIÓN Y GENERACIÓN DEL CÓDIGO FUENTE

5.1 Configuración del generador

La transformación del modelo UML hacia código fuente se realizó mediante **Java Designer 5.4.00**, módulo incorporado en **Modelio 5.4.01**, compilación 202312071609, para arquitectura Windows x86_64.

Se utilizó el generador estándar proporcionado por Java Designer. Esta decisión no respondió a una omisión del proceso de configuración. En la instalación utilizada, el módulo no expone desde su interfaz archivos de plantilla editables por el usuario. Por tanto, no se efectuó una personalización artificial que no pudiera reproducirse posteriormente.

La configuración se controló mediante los elementos que Java Designer interpreta directamente desde el modelo: estereotipos, tipos de datos, firmas de operaciones, multiplicidades, nombres de rol y visibilidad. Para identificar las clases generables se aplicó el estereotipo:

«JavaClass»

La configuración utilizada se resume en la Tabla 5.1.

Cuadro 5.1: Configuración del generador Java Designer

Parámetro	Configuración aplicada
Entorno de modelado	Modelio 5.4.01.
Generador	Java Designer 5.4.00, generador estándar del módulo.
Lenguaje objetivo	Java.
Estereotipo de generación	«JavaClass».
Directorio de salida	C:/Users/USER/modelio/workspace/Ing/src.
Convención de nombres	Cada archivo conserva el nombre de su clase UML y utiliza la extensión <code>.java</code> .
Fuente autoritativa	El modelo UML construido en Modelio.
Política de regeneración	Los cambios estructurales se corrigen en el modelo y después se regenera el archivo correspondiente.
Código manual	Se mantiene separado del código generado; únicamente <code>Main.java</code> fue escrito manualmente.

Debido a que no se personalizaron plantillas, se creó el archivo `plantillas/configuracion-java-designer.md` dentro del repositorio. Este archivo registra la versión del generador, los parámetros efectivos, las convenciones de nombres, las decisiones de mapeo, la política de regeneración y las limitaciones observadas. De esta manera, la configuración puede ser revisada y reproducida sin atribuir al módulo capacidades de edición que no fueron utilizadas.

5.2 Procedimiento de generación

La generación se ejecutó individualmente sobre las clases seleccionadas mediante el siguiente procedimiento:

1. Abrir el proyecto MundiPets en Modelio 5.4.01 y comprobar que Java Designer 5.4.00 estuviera habilitado.
2. Verificar la aplicación del estereotipo «JavaClass» en las clases destinadas a generación.
3. Seleccionar cada clase y ejecutar la opción **Java Designer** → **Generate**.
4. Comprobar la creación de los archivos `.java` en el directorio `src` y revisarlos mediante Visual Studio Code.

Las capturas de la habilitación del módulo, la aplicación del estereotipo, la ejecución del generador y los archivos obtenidos se presentan en los anexos.

El modelo UML se mantuvo como fuente principal de la estructura del código. Los errores relacionados con atributos, tipos, parámetros, operaciones, multiplicidades o nombres de rol fueron corregidos directamente en Modelio y no únicamente en los archivos Java.

El código escrito manualmente se mantuvo separado del código producido por Java Designer. No se evaluó la conservación automática de bloques protegidos ni de modificaciones manuales dentro de archivos regenerados.

5.3 Transformación UML a código Java

El proceso realizado corresponde a una transformación **Modelo a Texto (M2T)**, debido a que los elementos estructurados del modelo UML fueron convertidos en archivos textuales con extensión `.java`.

La correspondencia principal observada se presenta en la Tabla 5.2.

Cuadro 5.2: Decisiones de mapeo entre UML y Java

Elemento UML	Decisión de mapeo aplicada
Clase con «JavaClass»	Se genera una declaración de clase y un archivo <code>NombreClase.java</code> .
Atributo <code>string</code>	Se transforma en un campo Java de tipo <code>String</code> .
Atributo <code>integer</code>	Se transforma en un campo Java de tipo <code>int</code> .
Atributo <code>date</code>	Se transforma en <code>Date</code> con la importación correspondiente.
Operación UML	Se genera un método con nombre, parámetros y tipo de retorno.
Parámetro de dominio	Se utiliza la clase correspondiente, por ejemplo <code>Mascota mascota</code> , <code>Vacuna vacuna</code> o <code>DocumentoVeterinario documento</code> .
Multiplicidad <code>0..1</code> o <code>1</code>	Se genera una referencia simple a la clase relacionada.
Multiplicidad <code>0..*</code> o <code>*</code>	Se genera una colección tipada, por ejemplo <code>List<ValidacionMedica></code> .
Nombre de rol	Se utiliza como nombre del campo asociado; los roles múltiples se definieron en plural.
Agregación o composición	Se representa estructuralmente mediante referencias o colecciones. La gestión del ciclo de vida no es implementada automáticamente.

Java Designer produjo declaraciones de clases, atributos, operaciones, referencias y colecciones derivadas de las multiplicidades. El modelo actual no incorpora relaciones de generalización; por ello, el código generado no utiliza `extends` entre `Propietario` y `Veterinario`.

Los cuerpos de varias operaciones permanecieron vacíos, debido a que el generador produce principalmente la estructura inicial de las clases. La lógica de negocio, persistencia, interfaz gráfica y conexión con bases de datos deben implementarse manualmente.

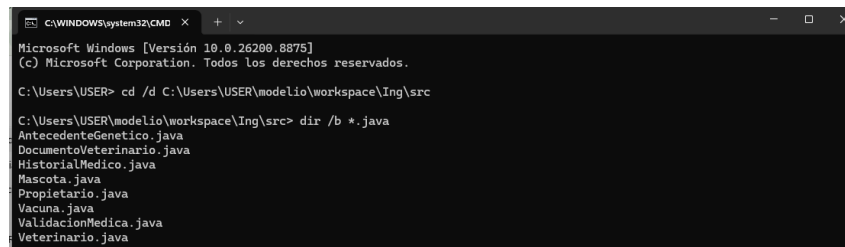
La compilación, ejecución y verificación de los archivos obtenidos se documentan en el capítulo siguiente.

6 VERIFICACIÓN DEL SOFTWARE GENERADO

6.1 Proceso de compilación

La verificación se realizó desde la consola de Windows en el directorio donde Java Designer almacenó el código generado (C:/Users/USER/modelio/workspace/Ing/src), confirmando con `javac -version` la disponibilidad de `javac 17.0.19`. Este compilador no intervino en la generación, realizada por Java Designer 5.4.00; se utilizó únicamente para verificar la validez sintáctica de los archivos obtenidos.

La compilación inicial mediante `javac *.java` permitió detectar parámetros sin nombre, tipos incorrectos y extremos de asociación incompletos, incluida una declaración sin nombre de campo (`private Propietario ;`). Las inconsistencias se corrigieron directamente en el modelo UML y las clases afectadas fueron regeneradas, manteniendo el modelo como fuente principal del código. Tras las correcciones, `javac *.java` no reportó errores y produjo ocho archivos compilados: `AntecedenteGenetico.class`, `DocumentoVeterinario.class`, `HistorialMedico.class`, `Mascota.class`, `Propietario.class`, `Vacuna.class`, `ValidacionMedica.class` y `Veterinario.class`.



```
C:\WINDOWS\system32\cmd
Microsoft Windows [Versión 10.0.26280.8875]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USER> cd /d C:\Users\USER\modelio\workspace\Ing\src

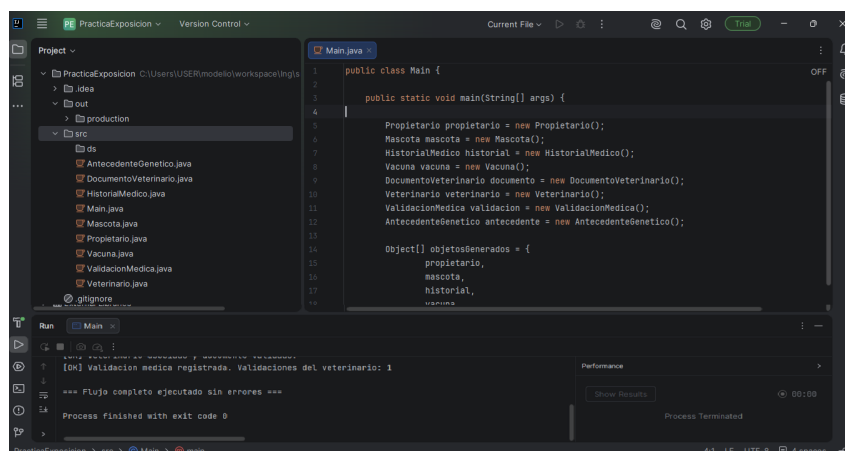
C:\Users\USER\modelio\workspace\Ing\src> dir /b *.java
AntecedenteGenetico.java
DocumentoVeterinario.java
HistorialMedico.java
Mascota.java
Propietario.java
Vacuna.java
ValidacionMedica.java
Veterinario.java
```

Figura 6.1: Compilación exitosa de las ocho clases Java generadas mediante Java Designer.

La evidencia confirma la correspondencia entre las ocho clases UML, los ocho archivos `.java` y los ocho archivos `.class` obtenidos.

6.2 Ejecución de una unidad demostrable

Las clases generadas representan la estructura del dominio pero no incluyen un método principal, por lo que se creó manualmente la clase `Main.java`, separada del código producido por Java Designer. Esta unidad instanció las ocho clases (`Propietario`, `Mascota`, `HistorialMedico`, `Vacuna`, `DocumentoVeterinario`, `Veterinario`, `ValidacionMedica` y `AntecedenteGenetico`), confirmó el mensaje `Total de clases verificadas: 8` y terminó con `Process finished with exit code 0`.



```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         Propietario propietario = new Propietario();
6         Mascota mascota = new Mascota();
7         HistorialMedico historial = new HistorialMedico();
8         Vacuna vacuna = new Vacuna();
9         DocumentoVeterinario documento = new DocumentoVeterinario();
10        Veterinario veterinario = new Veterinario();
11        ValidacionMedica validacion = new ValidacionMedica();
12        AntecedenteGenetico antecedente = new AntecedenteGenetico();
13
14        Object[] objetosGenerados = {
15            propietario,
16            mascota,
17            historial,
18            vacuna
19        }
20    }
21 }
```

Run Main

[OK] Validacion medica registrada. Validaciones del veterinario: 1

=== Flujo completo ejecutado sin errores ===

Process finished with exit code 0

Figura 6.2: Ejecución de las ocho clases mediante la unidad demostrable `Main.java`.

La clase `Main.java` corresponde a código manual y no se contabiliza dentro de las ocho

clases generadas.

6.3 Resultados y correspondencia modelo–código

La verificación no se limitó a contar los archivos generados. También se compararon las clases, atributos, operaciones y asociaciones definidos en el modelo UML con las declaraciones producidas por Java Designer.

La Tabla 6.1 presenta la trazabilidad entre elementos concretos del modelo y su representación en Java.

Cuadro 6.1: Correspondencia detallada entre el modelo UML y el código Java

Clase / archivo	Elemento definido en UML	Representación verificada en Java
Propietario	idPropietario:integer; operación registrarMascota(Mascota); asociación mascotas[0..*]	private int idPropietario; método con parámetro Mascota; colección List<Mascota>mascotas en Propietario.java.
Mascota	idMascota:integer, fechaNacimiento:date y microchip:string	Campos int idMascota, Date fechaNacimiento y String microchip en Mascota.java.
HistorialMedico	fechaRegistro:date; operaciones registrarVacuna(Vacuna) y adjuntarDocumento(DocumentoVeterinario)	Campo Date fechaRegistro y métodos con parámetros Vacuna y DocumentoVeterinario en HistorialMedico.java.
Vacuna	idVacuna:integer, fechaAplicacion:date y fechaVencimiento:date	Campos int idVacuna, Date fechaAplicacion y Date fechaVencimiento en Vacuna.java.
DocumentoVeterinario	idDocumento:integer, rutaArchivo:string; operaciones validarDocumento() y cargarArchivo()	Campos int idDocumento, String rutaArchivo y métodos correspondientes en DocumentoVeterinario.java.
Veterinario	numeroLicencia:string; asociación múltiple con ValidacionMedica	Campo String numeroLicencia y colección List<ValidacionMedica> en Veterinario.java.
ValidacionMedica	idValidacion:integer, estado:string; operaciones emitirObservacion(String) y cambiarEstado(String)	Campos int idValidacion, String estado y métodos con parámetros String en ValidacionMedica.java.
AntecedenteGenetico	idAntecedente:integer, descripcion:string y tipo:string	Campos int idAntecedente, String descripcion y String tipo en AntecedenteGenetico.java.

La comparación confirmó una correspondencia verificable entre las ocho clases del modelo, sus atributos tipados, sus operaciones y los ocho archivos Java generados. Las asociaciones con multiplicidad múltiple se representaron mediante colecciones tipadas, mientras que las multiplicidades máximas de uno produjeron referencias simples.

6.4 Limitaciones encontradas

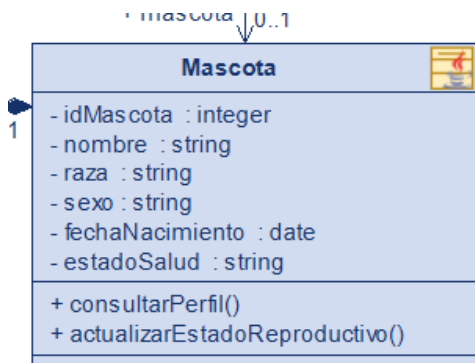
- Java Designer genera principalmente la estructura de las clases, pero no implementa completamente la lógica de negocio.
- La ejecución requiere una clase principal escrita manualmente.
- La persistencia, la interfaz gráfica y la conexión con bases de datos deben incorporarse posteriormente.
- La auditoría automática debe complementarse con revisión manual, compilación y ejecución del código.

En consecuencia, Java Designer facilita la trazabilidad estructural entre UML y Java, pero no sustituye la implementación manual necesaria para obtener una aplicación completamente funcional.

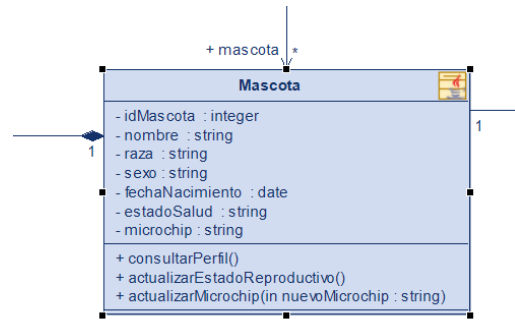
7 ROUNDTRIP ENGINEERING

7.1 Modificación controlada del modelo UML

Para comprobar el cierre controlado del ciclo modelo–código, se incorporó a la clase `Mascota` el atributo `microchip : string` y la operación `actualizarMicrochip(in nuevoMicrochip:string):void`, que permiten almacenar y actualizar el identificador electrónico de la mascota. Antes del cambio, la clase contenía únicamente los atributos `idMascota`, `nombre`, `raza`, `sexo`, `fechaNacimiento` y `estadoSalud`, y las operaciones `consultarPerfil()` y `actualizarEstadoReproductivo()`.



(a) Clase `Mascota` antes del cambio.



(b) Clase `Mascota` después del cambio.

Figura 7.1: Modelo UML antes y después de incorporar el atributo `microchip` y la operación `actualizarMicrochip`.

La modificación se considera un cambio estructural compuesto porque afecta simultáneamente un atributo, una operación, un parámetro, un tipo de retorno y la estructura del archivo Java generado. Los elementos se incorporaron directamente en Modelio, manteniendo el modelo UML como fuente principal del código.

7.2 Regeneración del código fuente

Después de modificar la clase `Mascota`, se ejecutó nuevamente `Java Designer` → `Generate` sobre el archivo `Mascota.java` almacenado en `C:/Users/USER/modelio/workspace/Ing/src`. El atributo UML se transformó en el campo `private String microchip`; y la operación se transformó en el método:

```

public void actualizarMicrochip(
    final String nuevoMicrochip) {
}
  
```

El cuerpo del método permaneció vacío porque `Java Designer` genera principalmente la estructura de las operaciones; ni el campo ni la firma fueron escritos manualmente. Tras la regeneración, el código se compiló nuevamente mediante `javac *.java` sin errores, confirmando que el nuevo atributo, operación y parámetro fueron transformados en estructuras Java sintácticamente válidas.

7.3 Comparación antes y después

La comparación de `Mascota.java` confirma que los cambios del modelo UML se trasladaron al código fuente sin intervención manual.

(a) Código antes de la regeneración.

(b) Código después de la regeneración.

Figura 7.2: Código de `Mascota.java` antes y después de generar el atributo y la operación incorporados en el modelo UML.

La Tabla 7.1 presenta la trazabilidad del cambio realizado.

Cuadro 7.1: Comparación del cambio realizado durante el roundtrip

Elemento	Antes del cambio	Después del cambio
Atributo UML	No existía el atributo <code>microchip</code> .	Se incorporó <code>microchip:string</code> .
Operación UML	No existía una operación para actualizar el identificador.	Se incorporó <code>actualizarMicrochip(in nuevoMicrochip:string):void</code> .
Campo Java	No existía un campo para almacenar el <code>microchip</code> .	Se generó <code>private String microchip;</code> .
Método Java	No existía un método de actualización.	Se generó <code>actualizarMicrochip(String nuevoMicrochip)</code> .
Parámetro	No existía un parámetro relacionado con el <code>microchip</code> .	Se generó el parámetro <code>final String nuevoMicrochip</code> .
Intervención manual	No se modificó directamente el archivo generado.	El cambio se realizó en Modelio y se regeneró <code>Mascota.java</code> .
Compilación	El código anterior compilaba correctamente.	El código regenerado volvió a compilar sin errores.

La evidencia demuestra la trazabilidad **atributo y operación UML → campo, método y parámetro Java**: un cierre controlado del ciclo modelo–código, en el que la modificación se realizó en Modelio y los nuevos elementos aparecieron en `Mascota.java` sin ser incorporados manualmente.

7.4 Manejo del código manual

La evidencia presentada no constituye una sincronización bidireccional completa, ya que el código Java no fue modificado para actualizar posteriormente el modelo UML. En la configuración estándar de Java Designer 5.4.00 no se comprobó la conservación automática de bloques protegidos, por lo que las ocho clases producidas se trataron como archivos generados y reemplazables. La clase `Main.java` y cualquier lógica manual se mantuvieron separadas del código generado, evitando que una regeneración sobrescriba la implementación manual.

8 REPOSITORIO GITHUB Y TRABAJO COLABORATIVO

8.1 Organización del repositorio

Los artefactos desarrollados durante la aplicación del enfoque MDD se almacenaron en un repositorio público de GitHub. Este recurso permitió centralizar el modelo UML, el código generado, el informe en LaTeX y las evidencias de generación, compilación, ejecución y roundtrip.

El repositorio se encuentra disponible en:

PFC-MundiPets-MDD-EquipoE

La estructura principal del repositorio se resume en la Tabla 8.1.

Cuadro 8.1: Estructura principal del repositorio

Elemento	Contenido
README.md	Descripción, integrantes, herramientas e instrucciones generales.
modelo/	Proyecto nativo de Modelio y diagrama UML.
plantillas/	Archivo README.md que documenta la configuración efectiva del generador (módulo Java Designer 5.4.00), justificando por qué no existen plantillas de texto editables al tratarse de un módulo compilado, e indicando el entorno base, fuente oficial y licencia del módulo utilizado.
codigo-generado/	Archivos Java producidos mediante Java Designer.
documentacion/	Informe en LaTeX, bibliografía y PDF final.
evidencias/	Capturas de auditoría, generación, compilación y roundtrip.

8.2 Reparto del trabajo por integrante

Nota: esta sección se encuentra pendiente de completar con el historial real de commits del repositorio antes de la entrega final. La rúbrica exige, para cada integrante, tareas concretas y una lista de commits atribuibles con su URL en GitHub; no debe entregarse en blanco ni con datos supuestos.

Cuadro 8.2: Reparto del trabajo por integrante (*pendiente de completar con datos reales*)

Integrante	Fases lideradas / tareas concretas	Commits atribuibles (URL)
Gary Alejandro Morales Sánchez	<i>Completar modelado UML, verificación de código, roundtrip.</i>	<i>[Completar con URLs de commits, ej.: github.com/.../commit/abcd123]</i>
Jimmy Samuel Nieves Sánchez	<i>Completar onfiguración de Java Designer, marco teórico, generación, compilación, documentación.</i>	<i>[Completar con URLs de commits]</i>

Los productos principales fueron revisados de manera conjunta antes de su incorporación al repositorio y al documento final.

8.3 Evidencias de colaboración

La participación del equipo se demuestra mediante el historial de commits, los archivos modificados y la sección de colaboradores de GitHub. Las capturas de la estructura del repositorio, los commits y las contribuciones individuales se presentan en los anexos.

9 CONCLUSIONES

1. La aplicación del enfoque de Desarrollo Dirigido por Modelos permitió utilizar el diagrama de clases UML de MundiPets como artefacto principal para obtener estructuras de código Java. Esto estableció una relación verificable entre el diseño del subsistema y los archivos generados.
2. Modelio 5.4.01, mediante el módulo Java Designer 5.4.00, permitió transformar las clases, atributos, operaciones, asociaciones y relaciones de generalización del modelo UML en elementos equivalentes del lenguaje Java. Para realizar esta transformación se utilizó el estereotipo «**JavaClass**».
3. La auditoría automática de Modelio no reportó errores, advertencias ni recomendaciones. Sin embargo, la compilación permitió identificar problemas que no fueron detectados por la validación automática, como parámetros sin nombre y extremos de asociaciones incompletos. Esto evidencia la necesidad de complementar la auditoría del modelo con una revisión manual y con la compilación del código.
4. Después de corregir las definiciones del modelo UML y regenerar las clases, la compilación mediante `javac *.java` terminó sin errores y produjo ocho archivos `.class`. Este resultado confirmó que el código estructural generado era válido para el compilador de Java.
5. La calidad del código producido depende directamente de la precisión del modelo de origen. Los tipos de datos, nombres de parámetros, multiplicidades y nombres de rol deben definirse correctamente en UML, debido a que cualquier inconsistencia puede trasladarse al archivo Java generado.
6. Java Designer facilita la automatización de la estructura inicial del software, pero no genera una aplicación completamente funcional. La lógica de negocio, persistencia, interfaz de usuario, conexión con bases de datos y ejecución mediante un método `main` requieren implementación manual posterior.
7. El uso del repositorio GitHub permitió organizar el modelo nativo, el código generado, el informe y las evidencias del proceso, proporcionando trazabilidad sobre las actividades desarrolladas y los aportes realizados por los integrantes del equipo.

En conclusión, el proyecto demostró que el enfoque MDD permite reducir la codificación repetitiva y mantener correspondencia entre el modelo UML y el código Java. No obstante, la generación automática debe acompañarse de validación, compilación y revisión técnica para garantizar que los artefactos obtenidos sean consistentes y utilizables.

10 REFERENCIAS

- [1] D. C. Schmidt, “Model-Driven Engineering,” *IEEE Computer*, vol. 39, n.º 2, págs. 25-31, 2006.
- [2] M. Brambilla, J. Cabot y M. Wimmer, *Model-Driven Software Engineering in Practice*. Morgan & Claypool Publishers, 2017.
- [3] A. Bucchiarone, J. Cabot, R. F. Paige y A. Pierantonio, “Grand challenges in model-driven engineering: an analysis of the state of the research,” *Software and Systems Modeling*, vol. 19, págs. 5-13, 2020. DOI: 10.1007/s10270-019-00773-6
- [4] O. M. Group, *Model Driven Architecture Guide Revision 2.0*, OMG Document ormsc/14-06-01, 2014.
- [5] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, OMG Document formal/17-12-05, 2017.
- [6] Object Management Group. “Meta Object Facility (MOF), Version 2.5.1.” OMG File ID: formal/19-10-01, Object Management Group, visitado 22 de jul. de 2026. dirección: <https://www.omg.org/spec/MOF/2.5.1/PDF>
- [7] Object Management Group. “MOF Query/View/Transformation (QVT), Version 1.3.” OMG File ID: formal/16-06-03, Object Management Group, visitado 22 de jul. de 2026. dirección: <https://www.omg.org/spec/QVT/1.3/PDF>
- [8] O. M. Group, *MOF Model To Text Transformation Language Version 1.0*, OMG Document formal/2008-01-16, 2008.
- [9] Modelio Open Source. “Modelio Modeling Environment and Main Features,” Modelio Open Source, visitado 22 de jul. de 2026. dirección: <https://github.com/ModelioOpenSource/Modelio>
- [10] Modelio Open Source. “Extensions for Modelio: Java Designer,” Modelio Open Source, visitado 22 de jul. de 2026. dirección: <https://github.com/ModelioOpenSource/ExtensionsForModelio>
- [11] Modelio Open Source. “Modelio License: GNU General Public License Version 3,” Modelio Open Source, visitado 22 de jul. de 2026. dirección: <https://github.com/ModelioOpenSource/Modelio/blob/master/LICENSE>

11 ANEXOS

Los anexos reúnen las evidencias complementarias del proceso de modelado, generación, compilación, regeneración y trabajo colaborativo. Estos elementos respaldan los resultados presentados en los capítulos anteriores.

11.1 Diagrama UML completo

La Figura 11.1 presenta el diagrama de clases completo del subsistema de gestión de información de mascotas y control veterinario de MundiPets.

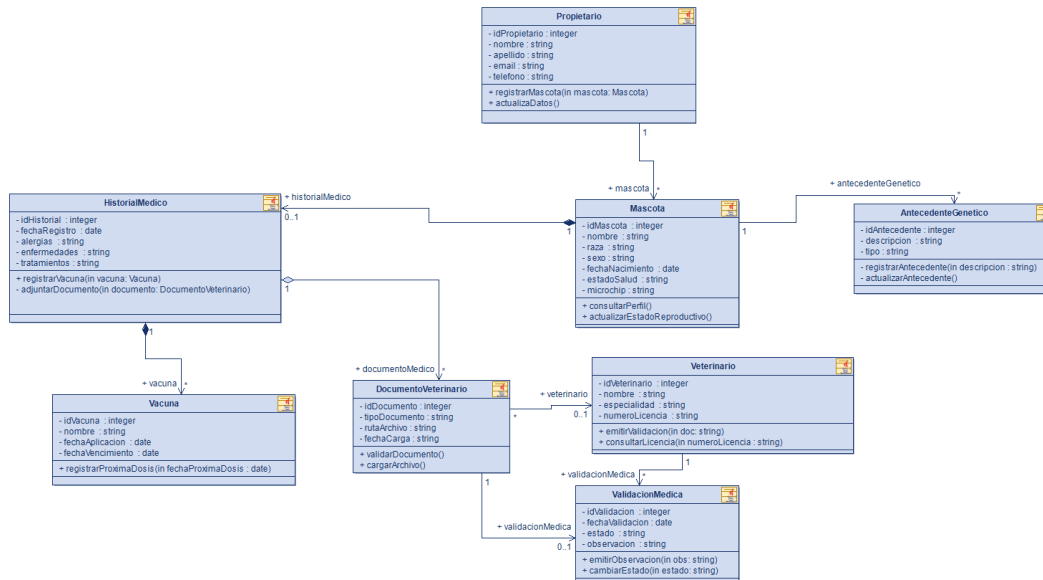


Figura 11.1: Diagrama de clases UML completo del subsistema MundiPets.

El archivo nativo del modelo desarrollado en Modelio se encuentra almacenado en el repositorio público del proyecto.

11.2 Capturas de la herramienta MDD

11.2.1 Configuración de Java Designer

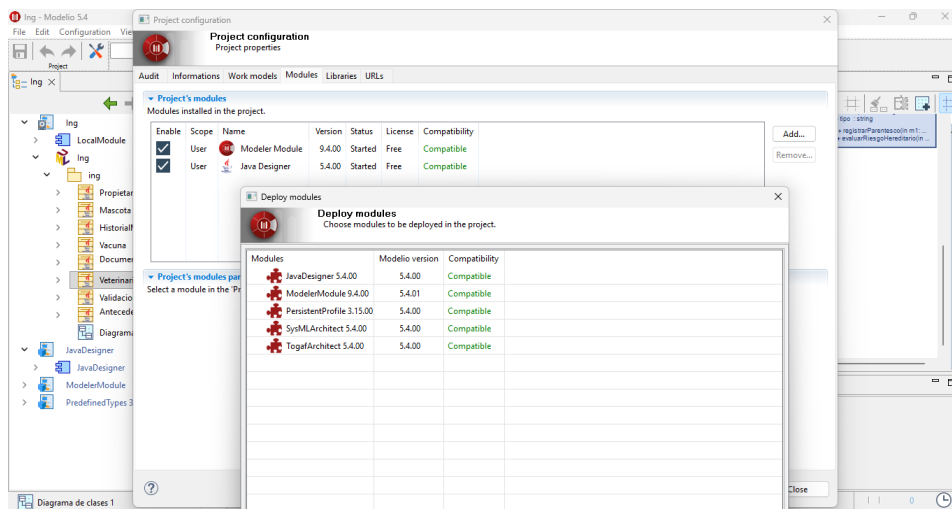


Figura 11.2: Módulo Java Designer 5.4.00 habilitado en Modelio 5.4.01.

11.2.2 Aplicación del estereotipo JavaClass

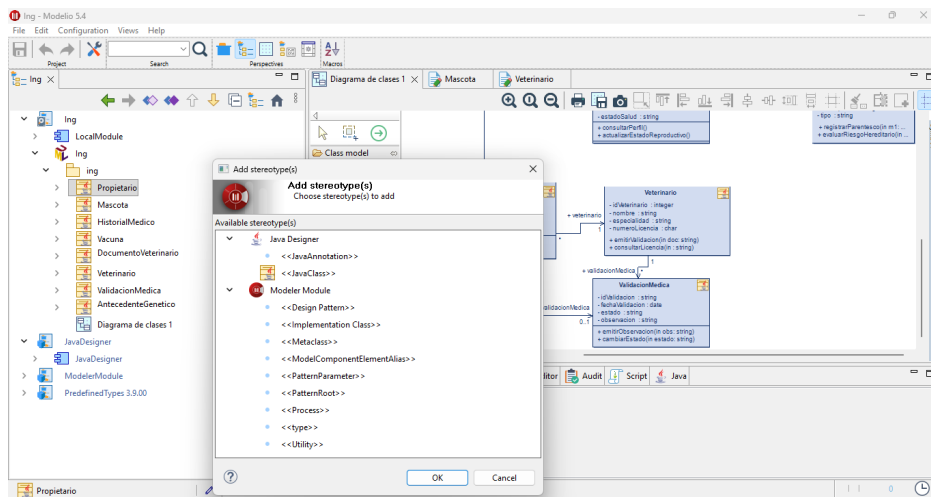


Figura 11.3: Aplicación del estereotipo «JavaClass» a una clase UML.

11.2.3 Validación automática del modelo

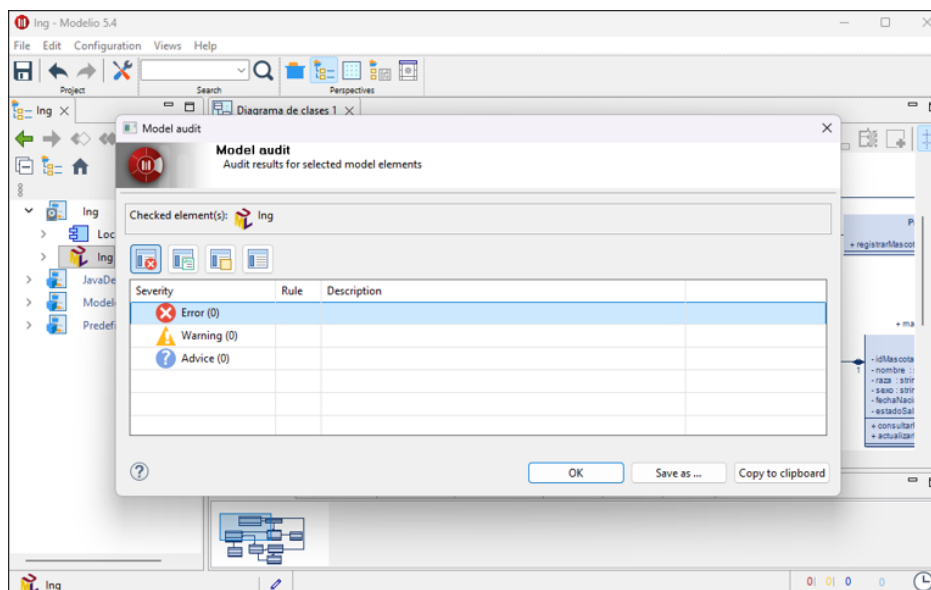


Figura 11.4: Resultado de la auditoría automática ejecutada en Modelio.

11.2.4 Ejecución del mecanismo de generación

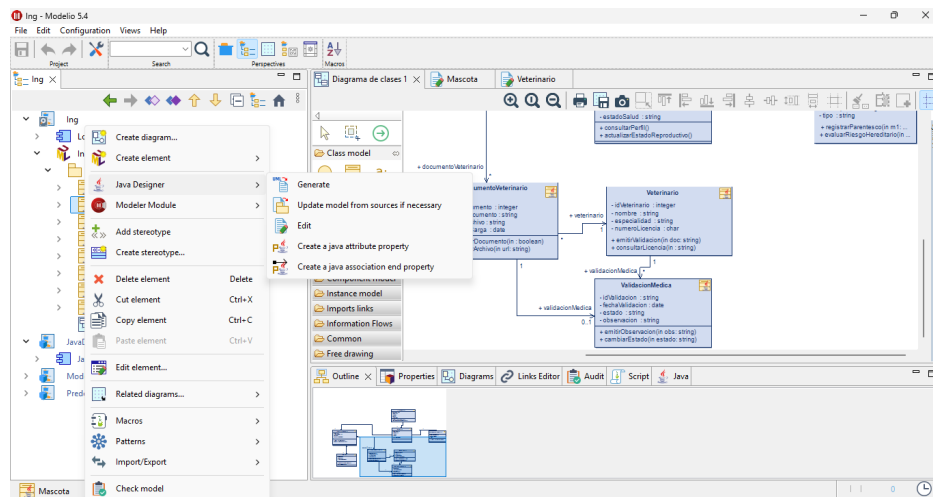


Figura 11.5: Ejecución de la opción Java Designer → Generate.

11.2.5 Archivos Java generados

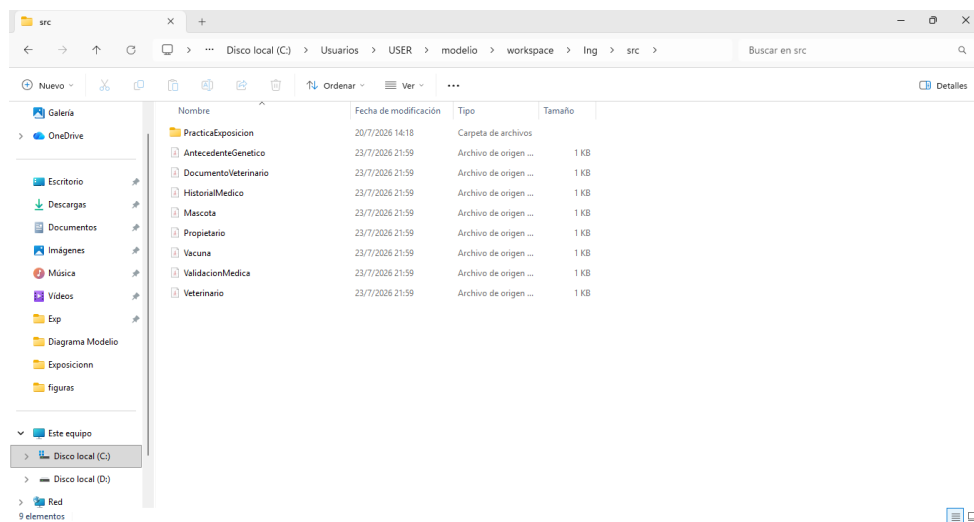


Figura 11.6: Archivos Java producidos por Java Designer en el directorio src.

11.3 Código fuente generado

Los siguientes listados presentan clases representativas obtenidas mediante Java Designer. El conjunto completo de archivos se encuentra disponible en el repositorio del proyecto.

11.3.1 Clase Mascota

```
1 import java.util.Date;
2
3 public class Mascota {
4     private int idMascota ;
5
6     private String nombre ;
7
8     private String raza ;
9
10    private String sexo;
11
12    private Date fechaNacimiento ;
13
14    private String estadoSalud ;
15
16    private String microchip;
17
18    public HistorialMedico historialMedico;
19
20    public void consultarPerfil() {
21    }
22
23    public void actualizarEstadoReproductivo() {
24    }
25
26 }
```

Listing 11.1: Código generado de la clase Mascota.

11.3.2 Clase Propietario

```
1
2
3 public class Propietario {
4     private int idPropietario;
5
6     private String nombre;
7
8     private String apellido;
9
10    private String email;
11
12    private String telefono;
13
14    public Mascota mascota;
15
16    public void registrarMascota() {
17    }
18
19    public void actualizaDatos() {
20    }
21
22 }
```

Listing 11.2: Código generado de la clase Propietario.

11.3.3 Clase Veterinario

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Veterinario {
5     private int idVeterinario ;
6
7     private String nombre ;
8
9     private String especialidad ;
10
11    private String numeroLicencia ;
12
13    public List<ValidacionMedica> validacionMedica = new ArrayList<ValidacionMedica> ();
14
15    public void emitirValidacion(final String doc) {
16    }
17
18    public void consultarLicencia(final String numeroLicencia ) {
19    }
20
21 }
```

Listing 11.3: Código generado de la clase Veterinario.

11.4 Evidencias de compilación y ejecución

11.4.1 Compilación del código generado

La compilación se realizó desde la consola de Windows mediante los comandos:

```
javac *.java
```

```
dir *.class
```

```

C:\WINDOWS\system32\CMD X + v
Microsoft Windows [Versión 10.0.26200.8875]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USER> cd /d C:\Users\USER\modelio\workspace\Ing\src

C:\Users\USER\modelio\workspace\Ing\src> dir /b *.java
AntecedenteGenetico.java
DocumentoVeterinario.java
HistorialMedico.java
Mascota.java
Propietario.java
Vacuna.java
ValidacionMedica.java
Veterinario.java

```

Figura 11.7: Compilación exitosa de las clases Java generadas.

La consola no reportó errores y mostró ocho archivos .class, correspondientes a las clases compiladas.

11.4.2 Ejecución de la unidad demostrable

```

Main.java
1 public class Main {
2
3     public static void main(String[] args) {
4
5         Propietario propietario = new Propietario();
6         Mascota mascota = new Mascota();
7         HistorialMedico historial = new HistorialMedico();
8         Vacuna vacuna = new Vacuna();
9         DocumentoVeterinario documento = new DocumentoVeterinario();
10        Veterinario veterinario = new Veterinario();
11        ValidacionMedica validacion = new ValidacionMedica();
12        AntecedenteGenetico antecedente = new AntecedenteGenetico();
13
14        Object[] objetosGenerados = {
15            propietario,
16            mascota,
17            historial,
18            vacuna,
19            documento,
20            veterinario,
21            validacion,
22            antecedente
23        };
24    }
25 }

```

```

Run
[OK] Validacion medica registrada. Validaciones del veterinario: 1
=== Flujo completo ejecutado sin errores ===
Process finished with exit code 0

```

Figura 11.8: Ejecución de la unidad demostrable PruebaMundiPets.

11.5 Evidencias del roundtrip

11.5.1 Modelo antes y después del cambio

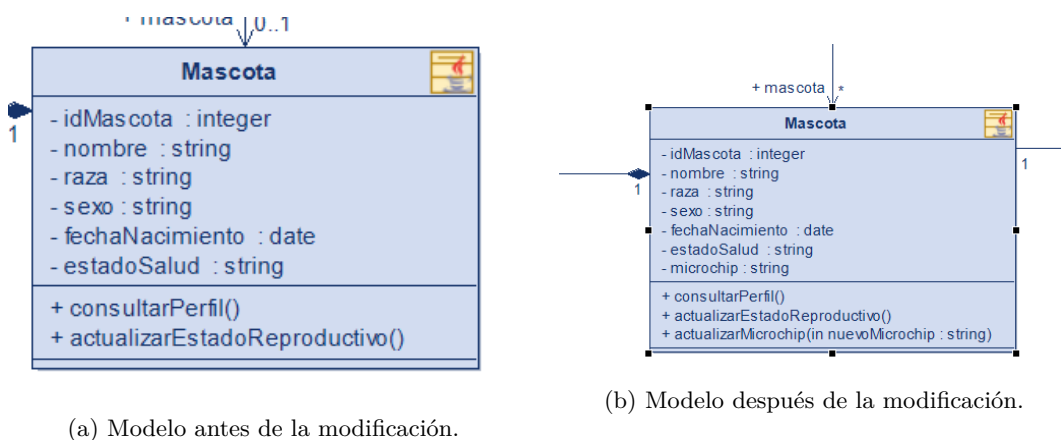
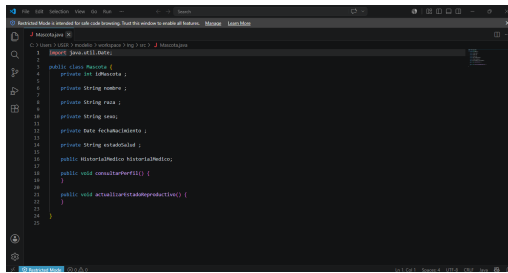
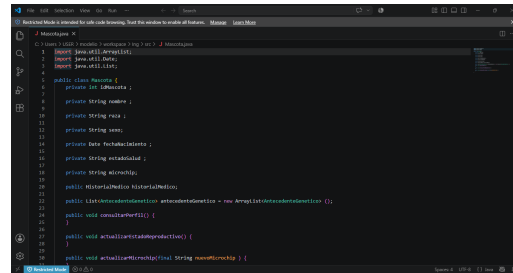


Figura 11.9: Comparación del modelo UML antes y después del cambio controlado.

11.5.2 Código antes y después de la regeneración



(a) Código antes de la regeneración.



(b) Código después de la regeneración.

Figura 11.10: Comparación del código Java antes y después de la regeneración.

11.6 Repositorio GitHub

El repositorio público utilizado para almacenar el modelo, código, documentación y evidencias se encuentra disponible en:

PFC-MundiPets-MDD-EquipoE

11.6.1 Estructura del repositorio

```
PFC-MundiPets-MDD-EquipoE/
|-- README.md
|-- docs/
|   |-- informe.tex
|   |-- informe.pdf
|   |-- referencias.bib
|   '-- figuras/           (diagramas y capturas)
|-- modelo/
|   |-- Ing.zip           (proyecto nativo de Modelio)
|   '-- perfiles/         (perfil UML aplicado)
|-- plantillas/
|   '-- README.md (justificación de la plantilla/configuración)
|-- codigo-generado/
|   '-- *.java           (8 clases generadas)
|-- evidencias/
|   '-- capturas/        (generación, compilación, verificación)
|-- exposicion/
|   |-- diapositivas.pdf
|   '-- README.md        (enlace al video)
'-- borradores/
    |-- MoralesGary/
    |   '-- BorradorMoralesSanchezGaryAlejandro.pdf
    '-- NievesJimmy/
        '-- BorradorNievesSanchezJimmySamuel.pdf
```

Figura 11.11: Estructura general del repositorio del proyecto.

11.6.2 Historial de commits



Figura 11.12: Historial de commits realizados durante el desarrollo.

11.6.3 Participación de los integrantes



Figura 11.13: Contribuciones registradas por los integrantes del equipo.